Continuous Integration (CI)

TL;DR
In größeren Projekten mit mehreren Teams werden die Beteiligten i.d.R. nur noch “ihre” Codestellen compilieren und testen. Dennoch ist es wichtig, das gesamte Projekt regelmäßig zu “bauen” und auch umfangreichere Testsuiten regelmäßig laufen zu lassen. Außerdem ist es wichtig, das in einer definierten Umgebung zu tun und nicht auf einem oder mehreren Entwicklerrechnern, die i.d.R. (leicht) unterschiedlich konfiguriert sind, um zuverlässige und nachvollziehbare Ergebnisse zu bekommen. Weiterhin möchte man auf bestimmte Ereignisse reagieren, wie etwa neue Commits im Git-Server, oder bei Pull-Requests möchte man vor dem Merge automatisiert sicherstellen, dass damit die vorhandenen Tests alle “grün” sind und auch die Formatierung etc. stimmt.
Dafür hat sich “Continuous Integration” etabliert. Hier werden die angesprochenen Prozesse regelmäßig auf einem dafür eingerichteten System durchgeführt. Aktivitäten wie Übersetzen, Testen, Style-Checks etc. werden in sogenannten “Pipelines” oder “Workflows” zusammengefasst und automatisiert durch Commits, Pull-Requests oder Merges auf dem Git-Server ausgelöst. Die Aktionen können dabei je nach Trigger und Branch unterschiedlich sein, d.h. man könnte etwa bei PR gegen den Master umfangreichere Tests laufen lassen als bei einem PR gegen einen Develop-Branch. In einem Workflow oder einer Pipeline können einzelne Aktionen wiederum von anderen Aktionen abhängen. Das Ergebnis kann man dann auf dem Server einsehen oder bekommt man komfortabel als Report per Mail zugeschickt.
Wir schauen uns hier exemplarisch GitHub Actions und GitLab CI/CD an. Um CI sinnvoll einsetzen zu können, benötigt man Kenntnisse über Build-Tools. “CI” tritt üblicherweise zusammen mit “CD” (Continuous Delivery) auf, also als “CI/CD”. Der “CD”-Teil ist nicht Gegenstand der Betrachtung in dieser Lehrveranstaltung.

Videos

- VL Continuous Integration
- Demo GitHub Actions
- Demo Demo GitLab CI/CD

Unterlagen

- PDF-Version

Motivation: Zusammenarbeit in Teams

Szenario

- Projekt besteht aus diversen Teilprojekten
- Verschiedene Entwicklungs-Teams arbeiten (getrennt) an verschiedenen Projekten
- Tester entwickeln Testsuiten für die Teilprojekte
- Tester entwickeln Testsuiten für das Gesamtprojekt

Manuelle Ausführung der Testsuiten reicht nicht

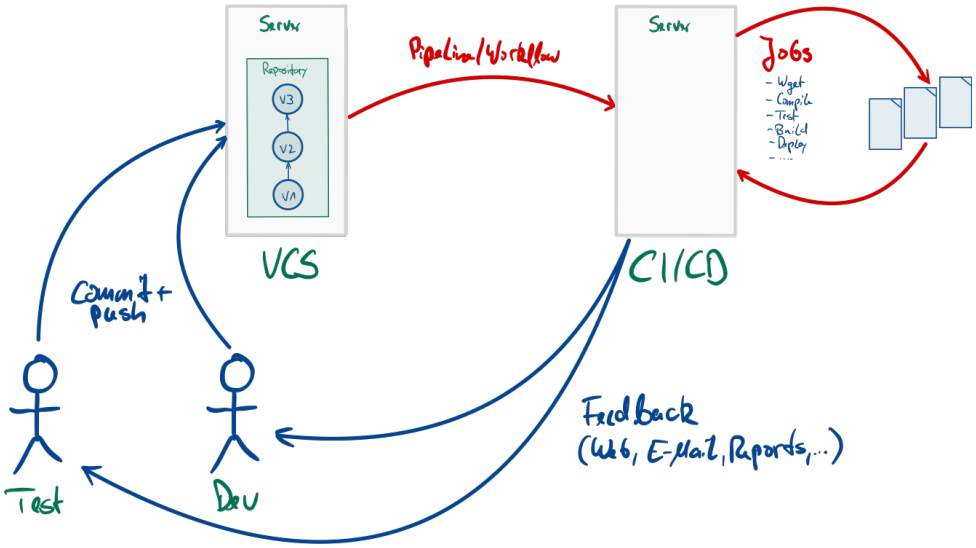
- Belastet den Entwicklungsprozess

- Keine (einheitliche) Veröffentlichung der Ergebnisse
- Keine (einheitliche) Eskalation bei Fehlern
- Keine regelmäßige Integration in Gesamtprojekt

Continuous Integration

- Regelmäßige, automatische Ausführung: Build und Tests
- Reporting
- Weiterführung der Idee: Regelmäßiges Deployment (*Continuous Deployment*)

Continuous Integration (CI)



Vorgehen

- Entwickler und Tester committen ihre Änderungen regelmäßig (Git, SVN, …)
- CI-Server arbeitet Build-Skripte ab, getriggert durch Events: Push-Events, Zeit/Datum, …
  - Typischerweise wird dabei:
    * Das Gesamtprojekt übersetzt (“gebaut”)
    * Die Unit- und die Integrationstests abgearbeitet
    * Zu festen Zeiten werden zusätzlich Systemtests gefahren
  - Typische weitere Builds: “Nightly Build”, Release-Build, …
  - Ergebnisse jeweils auf der Weboberfläche einsehbar (und per E-Mail)

Einige Vorteile

- Tests werden regelmäßig durchgeführt (auch wenn sie lange dauern oder die Maschine stark belasten)

- Es wird regelmäßig ein Gesamt-Build durchgeführt
- Alle Teilnehmer sind über aktuellen Projekt(-zu-)stand informiert

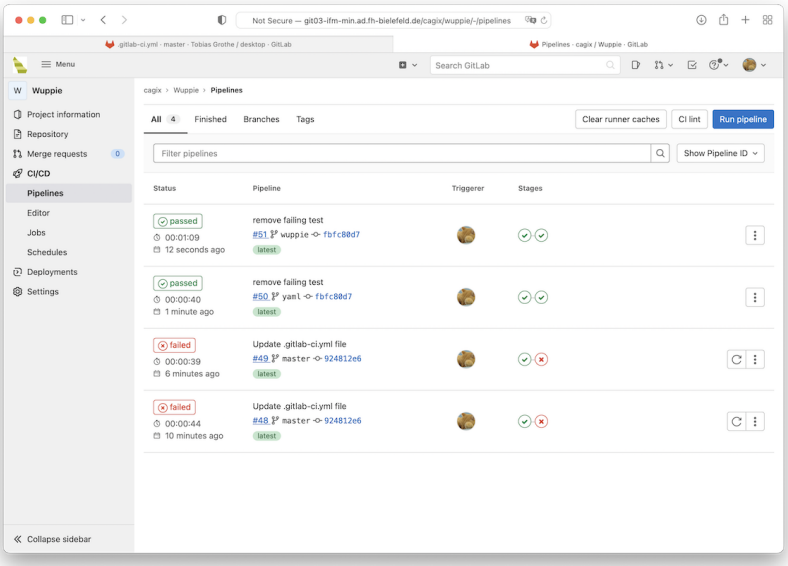
Beispiele für verbreitete CI-Umgebungen

- Jenkins
- GitLab CI/CD
- GitHub Actions und GitHub CI/CD
- Bamboo
- Travis CI

GitLab CI/CD

Siehe auch "Get started with Gitlab CI/CD". (Für den Zugriff wird VPN benötigt!)

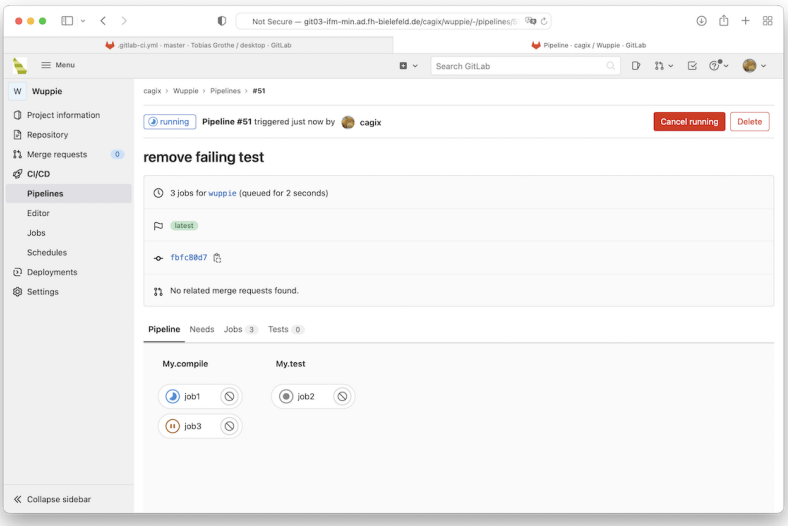
Übersicht über Pipelines



- In Spalte "Status" sieht man das Ergebnis der einzelnen Pipelines: "pending" (die Pipeline läuft gerade), "cancelled" (Pipeline wurde manuell abgebrochen), "passed" (alle Jobs der Pipeline sind sauber durchgelaufen), "failed" (ein Job ist fehlgeschlagen, Pipeline wurde deshalb abgebrochen)
- In Spalte "Pipeline" sind die Pipelines eindeutig benannt aufgeführt, inkl. Trigger (Commit und Branch)
- In Spalte "Stages" sieht man den Zustand der einzelnen Stages

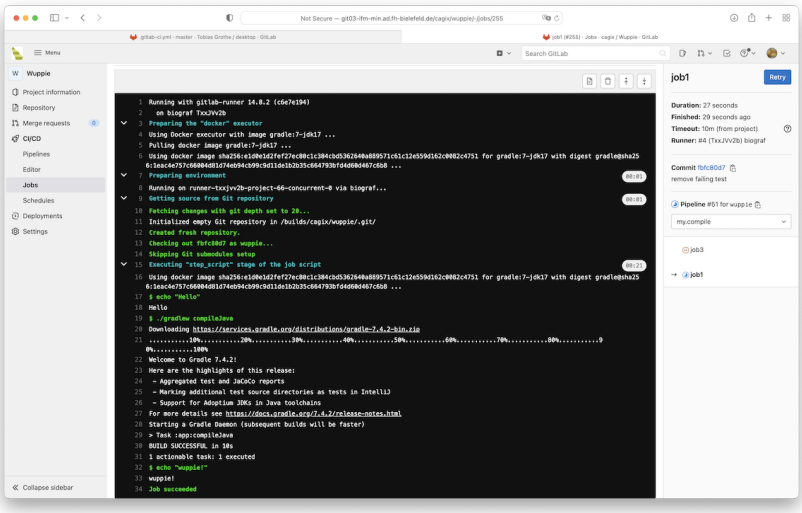
Wenn man mit der Maus auf den Status oder die Stages geht, erfährt man mehr bzw. kann auf eine Seite mit mehr Informationen kommen.

Detailansicht einer Pipeline



Wenn man in eine Pipeline in der Übersicht klickt, werden die einzelnen Stages dieser Pipeline genauer dargestellt.

Detailansicht eines Jobs



Wenn man in einen Job einer Stage klickt, bekommt man quasi die Konsolenausgabe dieses Jobs. Hier kann man ggf. Fehler beim Ausführen der einzelnen Skripte oder die Ergebnisse beispielsweise der JUnit-Läufe anschauen.

GitLab CI/CD: Konfiguration mit YAML-Datei

Datei `.gitlab-ci.yml` im Projekt-Ordner:

```yaml
stages:
    - my.compile
    - my.test

job1:
    script:
        - echo "Hello"
        - ./gradlew compileJava
        - echo "wuppie!"
    stage: my.compile
    only:
        - wuppie

job2:
    script: "./gradlew test"
    stage: my.test

job3:
    script:
        - echo "Job 3"
    stage: my.compile
```

Stages Unter `stages` werden die einzelnen Stages einer Pipeline definiert. Diese werden in der hier spezifizierten Reihenfolge durchgeführt, d.h. zuerst würde `my.compile` ausgeführt, und erst wenn alle Jobs in `my.compile` erfolgreich ausgeführt wurden, würde anschließend `my.test` ausgeführt.

Dabei gilt: Die Jobs einer Stage werden (potentiell) parallel zueinander ausgeführt, und die Jobs der nächsten Stage werden erst dann gestartet, wenn alle Jobs der aktuellen Stage erfolgreich beendet wurden.

Wenn keine eigenen `stages` definiert werden, kann man (lt. Doku) auf die Default-Stages `build`, `test` und `deploy` zurückgreifen. **Achtung**: Sobald man eigene Stages definiert, stehen diese Default-Stages *nicht* mehr zur Verfügung!

Jobs `job1`, `job2` und `job3` definieren jeweils einen Job.

- `job1` besteht aus mehreren Befehlen (unter `script`). Alternativ kann man die bei `job2` gezeigte Syntax nutzen, wenn nur ein Befehl zu bearbeiten ist.

  Die Befehle werden von GitLab CI/CD in einer Shell ausgeführt.

- Die Jobs `job1` und `job2` sind der Stage `my.compile` zugeordnet (Abschnitt `stage`). Einer Stage können mehrere Jobs zugeordnet sein, die dann parallel ausgeführt werden.

  Wenn ein Job nicht explizit einer Stage zugeordnet ist, wird er (lt. Doku) zur Default-Stage `test` zugewiesen. (Das geht nur, wenn es diese Stage auch gibt!)

- Mit `only` und `except` kann man u.a. Branches oder Tags angeben, für die dieser Job ausgeführt (bzw. nicht ausgeführt) werden soll.

Durch die Kombination von Jobs mit der Zuordnung zu Stages und Events lassen sich unterschiedliche Pipelines für verschiedene Zwecke definieren.

Hinweise zur Konfiguration von GitLab CI/CD

Im Browser in den Repo-Einstellungen arbeiten:

1. Unter `Settings > General > Visibility, project features, permissions` das `CI/CD` aktivieren
2. Prüfen unter `Settings > CI/CD > Runners`, dass unter `Available shared Runners` mind. ein shared Runner verfügbar ist (mit grün markiert ist)
3. Unter `Settings > CI/CD > General pipelines` einstellen:
   - `Git strategy`: `git clone`
   - `Timeout`: `10m`
   - `Public pipelines`: **`false`** (nicht angehakt)
4. YAML-File (`.gitlab-ci.yml`) in Projektwurzel anlegen, Aufbau siehe oben
5. Build-Skript erstellen, **lokal** lauffähig bekommen, dann in Jobs nutzen
6. Im `.gitlab-ci.yml` die relevanten Branches einstellen (s.o.)
7. Pushen, und unter `CI/CD > Pipelines` das Builden beobachten
   - in Status reinklicken und schauen, ob und wo es hakt
8. `README.md` anlegen in Projektwurzel (neben `.gitlab-ci.yml`), Markdown-Schnipsel aus `Settings > CI/CD > General pipelines > Pipeline status` auswählen und einfügen ....

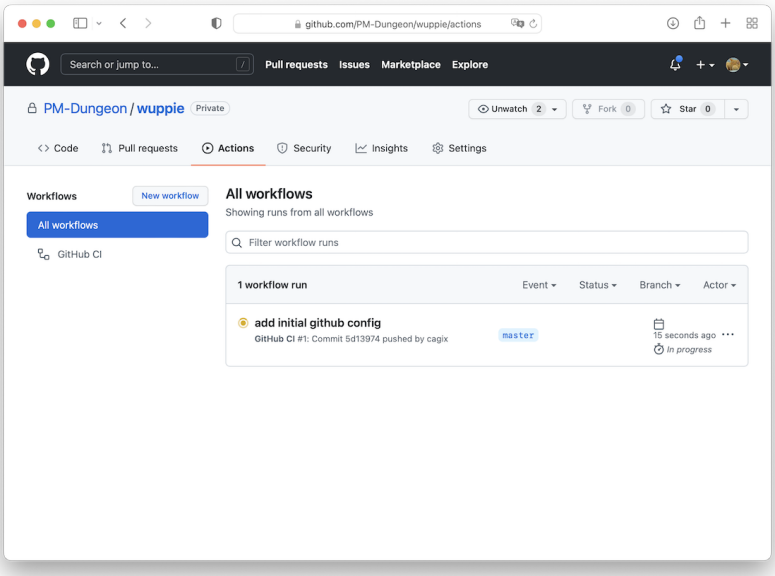
Optional:

9. Ggf. Schedules unter `CI/CD > Schedules` anlegen
10. Ggf. extra Mails einrichten: `Settings > Integrations > Pipeline status emails`

GitHub Actions

Siehe "GitHub Actions: Automate your workflow from idea to production" und auch "GitHub: CI/CD explained".

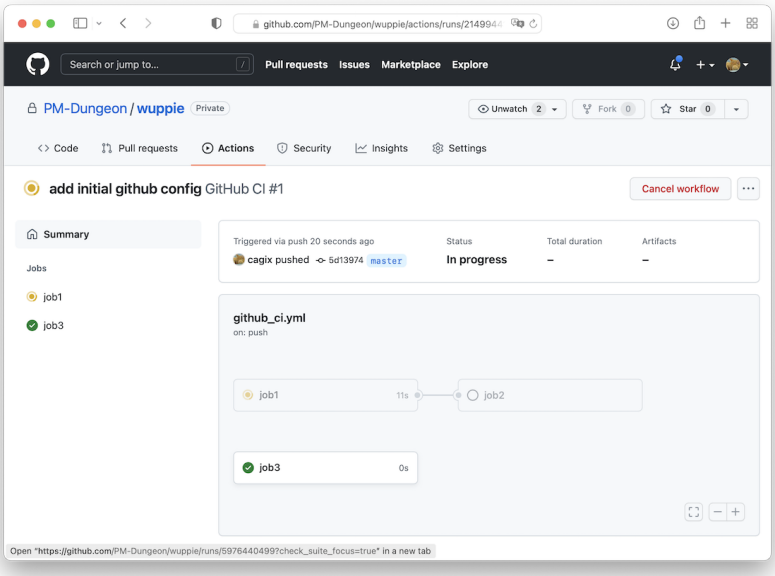
Übersicht über Workflows



Hier sieht man das Ergebnis der letzten Workflows. Dazu sieht man den Commit und den Branch, auf dem der Workflow gelaufen ist sowie wann er gelaufen ist. Über die Spalten kann man beispielsweise nach Status oder Event filtern.

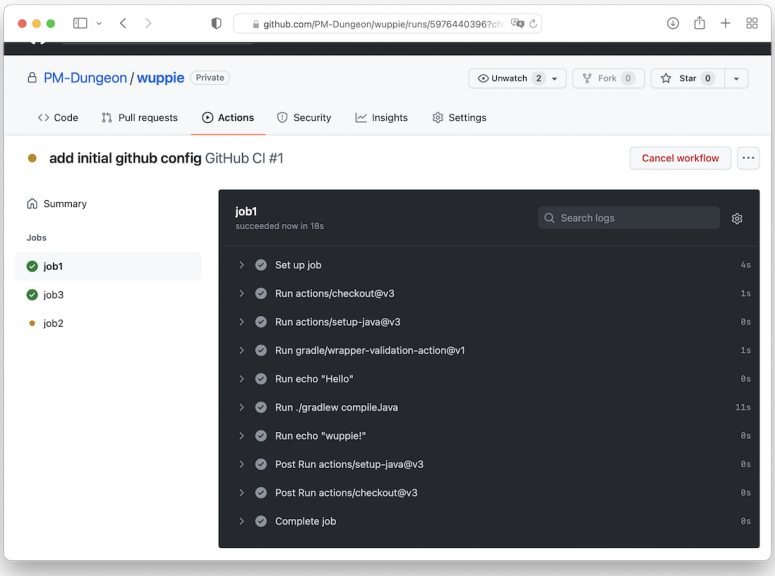
In der Abbildung ist ein Workflow mit dem Namen “GitHub CI” zu sehen, der aktuell noch läuft.

Detailansicht eines Workflows



Wenn man in einen Workflow in der Übersicht anklickt, werden die einzelnen Jobs dieses Workflows genauer dargestellt. “job3” ist erfolgreich gelaufen, “job1” läuft gerade, und “job2” hängt von “job1” ab, d.h. kann erst nach dem erfolgreichen Lauf von “job2” starten.

Detailansicht eines Jobs



Wenn man in einen Job anklickt, bekommt man quasi die Konsolenausgabe dieses Jobs. Hier kann man ggf. Fehler beim Ausführen der einzelnen Skripte oder die Ergebnisse beispielsweise der JUnit-Läufe anschauen.

GitHub Actions: Konfiguration mit YAML-Datei

Workflows werden als YAML-Dateien im Ordner `.github/workflows/` angelegt.

```yaml
name: GitHub CI

on:
  # push on master branch
  push:
    branches: [master]
  # manually triggered
  workflow_dispatch:

jobs:

  job1:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v3
        with:
```

```
18            java-version: '17'
19            distribution: 'temurin'
20        - uses: gradle/wrapper-validation-action@v1
21        - run: echo "Hello"
22        - run: ./gradlew compileJava
23        - run: echo "wuppie!"
24
25    job2:
26      needs: job1
27      runs-on: ubuntu-latest
28      steps:
29        - uses: actions/checkout@v4
30        - uses: actions/setup-java@v3
31          with:
32            java-version: '17'
33            distribution: 'temurin'
34        - uses: gradle/wrapper-validation-action@v1
35        - run: ./gradlew test
36
37    job3:
38      runs-on: ubuntu-latest
39      steps:
40        - run: echo "Job 3"
```

Workflowname und Trigger-Events Der Name des Workflows wird mit dem Eintrag `name` spezifiziert und sollte sich im Dateinamen widerspiegeln, also im Beispiel `.github/workflows/github_ci.yml`.

Im Eintrag `on` können die Events definiert werden, die den Workflow triggern. Im Beispiel ist ein Push-Event auf dem `master`-Branch definiert sowie mit `workflow_dispatch`: das manuelle Triggern (auf einem beliebigen Branch) freigeschaltet.

Jobs Die Jobs werden unter dem Eintrag `jobs` definiert: `job1`, `job2` und `job3` definieren jeweils einen Job.

- `job1` besteht aus mehreren Befehlen (unter `steps`), die auf einem aktuellen virtualisierten Ubuntu-Runner ausgeführt werden.

  Es wird zunächst das Repo mit Hilfe der Checkout-Action ausgecheckt (`uses: actions/checkout@v4`), das JDK eingerichtet/installiert (`uses: actions/setup-java@v3`) und der im Repo enthaltene Gradle-Wrapper auf Unversehrtheit geprüft (`uses: gradle/wrapper-validation-action@v1`).

  Die Actions sind vordefinierte Actions und im Github unter `github.com/` + Action zu finden, d.h. `actions/checkout` oder `actions/setup-java`. Actions können von jedermann definiert und bereitgestellt werden, in diesem Fall handelt es sich um von GitHub selbst im Namespace “actions” bereit gestellte direkt nutzbare Actions. Man kann Actions auch selbst im Ordner `.github/actions/` für das Repo definieren (Beispiel: plfa.github.io).

  Mit `run` werden Befehle in der Shell auf dem genutzten Runner (hier Ubuntu) ausgeführt.

- Die Jobs `job2` ist von `job1` abhängig und wird erst gestartet, wenn `job1` erfolgreich abgearbeitet ist.

Ansonsten können die Jobs prinzipiell parallel ausgeführt werden.

Durch die Kombination von Workflows mit verschiedenen Jobs und Abhängigkeiten zwischen Jobs lassen sich unterschiedliche Pipelines ("Workflows") für verschiedene Zwecke definieren.

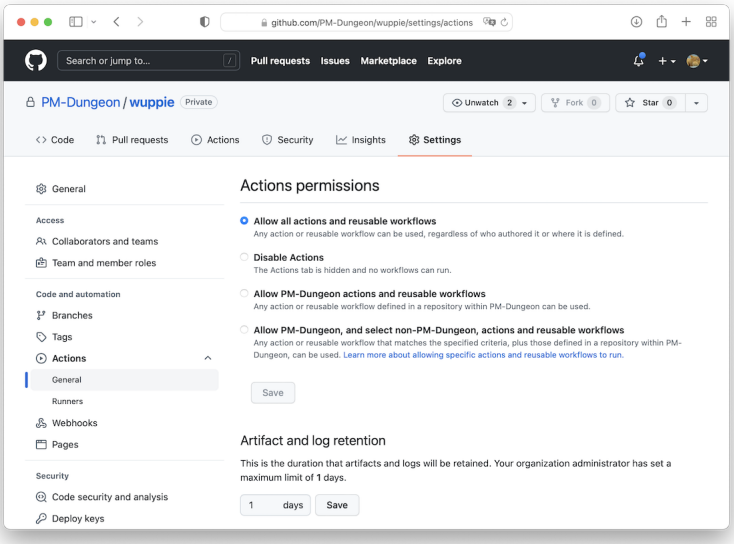
Es lassen sich auch andere Runner benutzen, etwa ein virtualisiertes Windows oder macOS. Man kann auch über einen "Matrix-Build" den Workflow auf mehreren Betriebssystemen gleichzeitig laufen lassen.

Man kann auch einen Docker-Container benutzen. Dabei muss man beachten, dass dieser am besten aus einer Registry (etwa von Docker-Hub oder aus der GitHub-Registry) "gezogen" wird, weil das Bauen des Docker-Containers aus einem Docker-File in der Action u.U. relativ lange dauert.

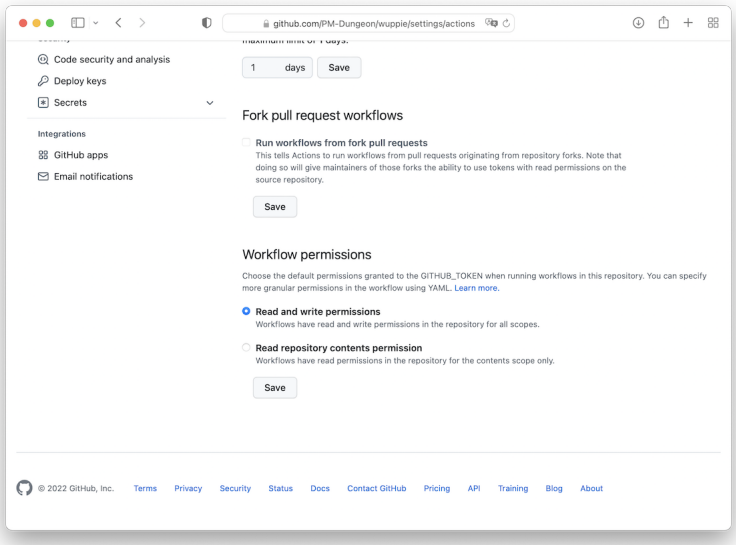
Hinweise zur Konfiguration von GitHub Actions

Im Browser in den Repo-Einstellungen arbeiten:

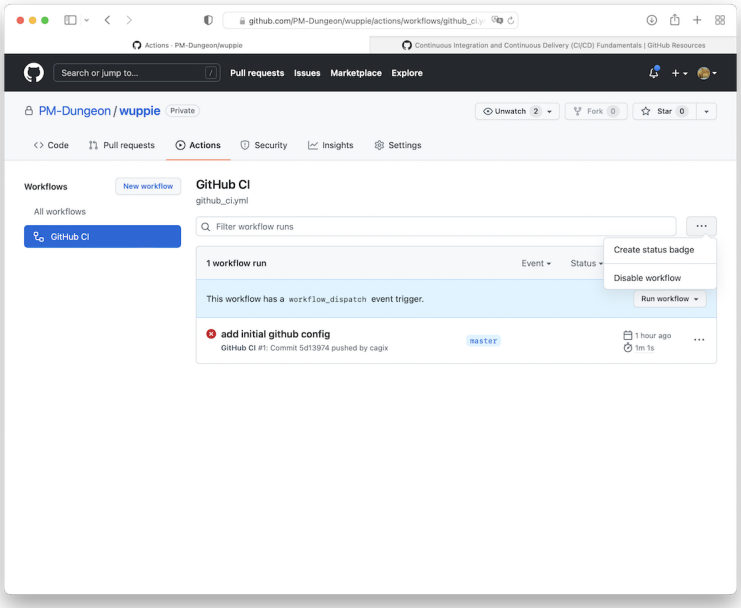
1. Unter `Settings` > `Actions` > `General` > `Actions permissions` die Actions aktivieren (Auswahl, welche Actions erlaubt sind)



2. Unter `Settings` > `Actions` > `General` > `Workflow permissions` ggf. bestimmen, ob die Actions das Repo nur lesen dürfen oder auch zusätzlich schreiben dürfen



3. Unter `Actions` > `<WORKFLOW>` den Workflow ggf. deaktivieren:



Wrap-Up

Überblick über Continuous Integration:

- Konfigurierbare Aktionen, die auf dem Gitlab-/GitHub-Server ausgeführt werden
- Unterschiedliche Trigger: Commit, Merge, …
- Aktionen können Branch-spezifisch sein
- Aktionen können von anderen Aktionen abhängen

Zum Nachlesen

- „Documentation Gitlab CI“ (o. J.)
- GitHub Inc. (2022)

Lernziele

- k2: Arbeitsweise von/mit CI

Challenges

Betrachten Sie erneut das Projekt Theatrical Players Refactoring Kata. Erstellen Sie für dieses Projekt einen GitHub-Workflow, der das Projekt kompiliert und die Testsuite ausführt (nur für den Java-Teil, den restlichen Code können Sie ignorieren).
Dabei soll das Ausführen der JUnit-Tests nur dann erfolgen, wenn das Kompilieren erfolgreich durchgeführt wurde. Der Workflow soll automatisch für Commits in den Hauptbranch sowie für Pull-Requests loslaufen. Es soll zusätzlich auch manuell aktivierbar sein.

Quellen

„Documentation Gitlab CI“. o. J. Zugegriffen 10. März 2022. http://git03-ifm-min.ad.hsbi.de/help/ci/.
GitHub Inc. 2022. „Documentation GitHub CI“. 2022. https://resources.github.com/ci-cd/.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** 2ce9f3f (Tooling: use original folder structure for generated pdf's, 2025-04-17)